

Analysis Report

Like

Share

860 people like this. [Sign Up](#) to see what your friends like.

Share this Report

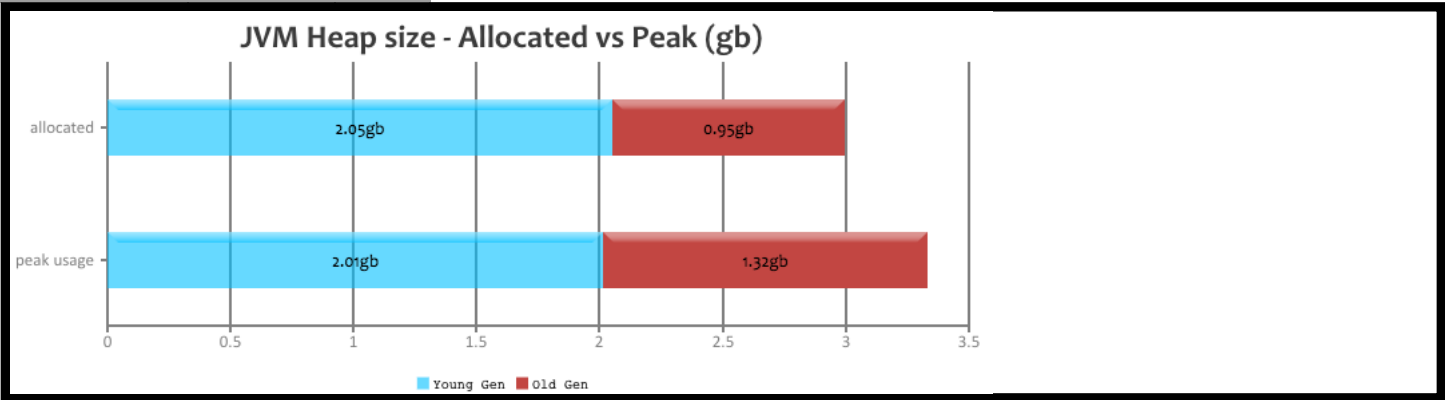
GC log file: **gc-2017_08_03-12_23.log.0**

Duration: **48 min 36 sec 306 ms**

 Congratulations! Your application's GC activity is healthy.

JVM Heap Size

Generation	Allocated 	Peak 
Young Generation	2.05 gb	2.01 gb
Old Generation	969 mb	1.32 gb
Total	3 gb	2.64 gb



Key Performance Indicators

(Important section of the report. To learn more about KPIs, [click here](https://blog.gceasy.io/2016/10/01/garbage-collection-kpi/) (<https://blog.gceasy.io/2016/10/01/garbage-collection-kpi/>))

1 Throughput  : 99.567%

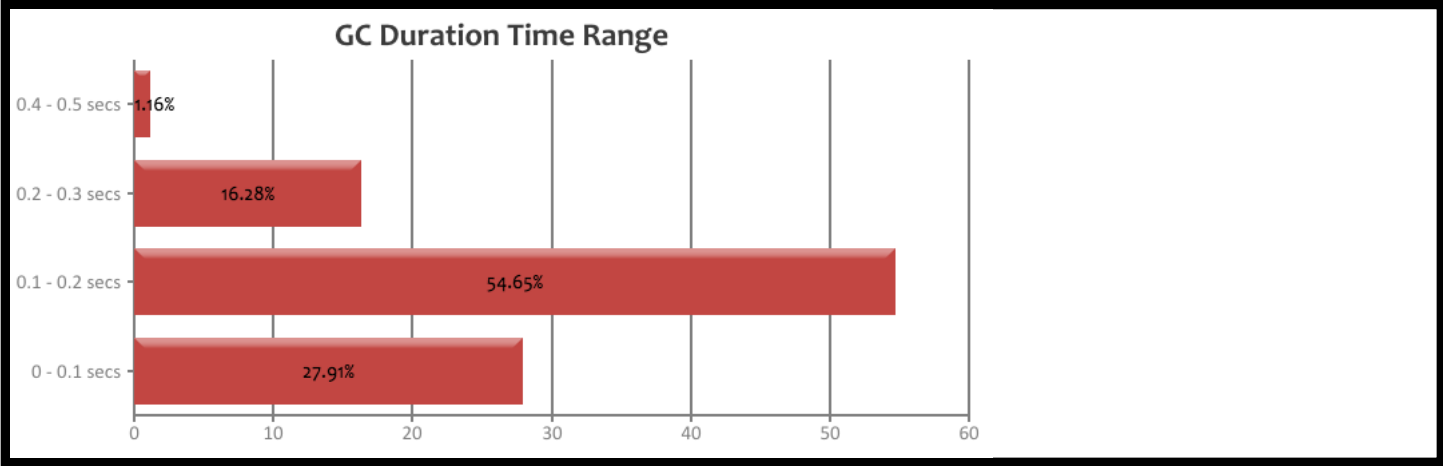
2 Latency:

Avg GC Time 	126 ms
Max GC Time 	470 ms

GC Duration Time Range 

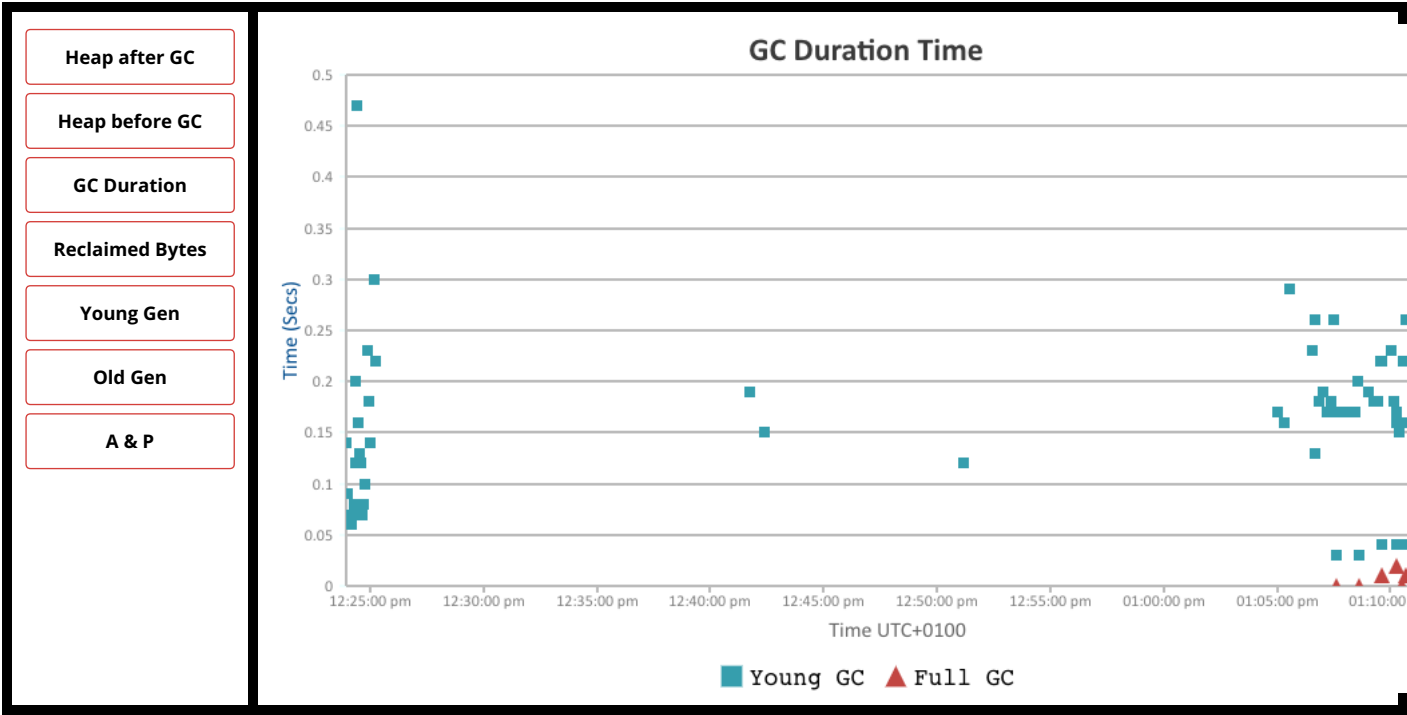
Duration (secs)	No. of GCs	Percentage
-----------------	------------	------------

0 - 0.1	24	27.907%
0.1 - 0.2	47	82.558%
0.2 - 0.3	14	98.837%
0.4 - 0.5	1	100.0%

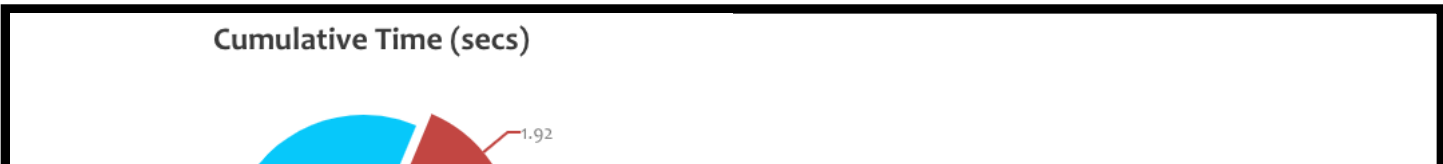


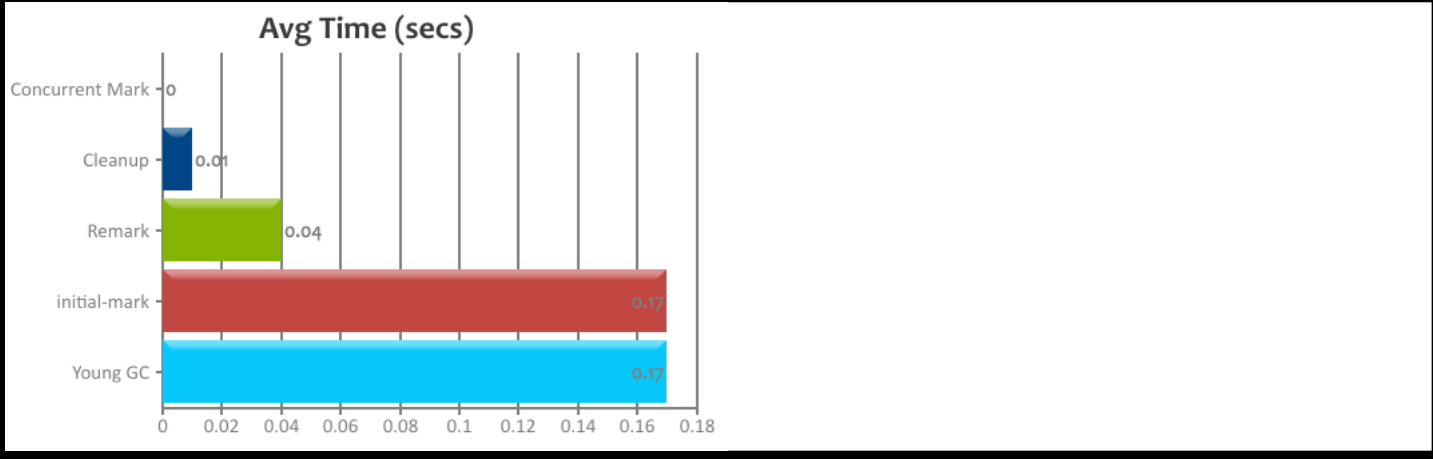
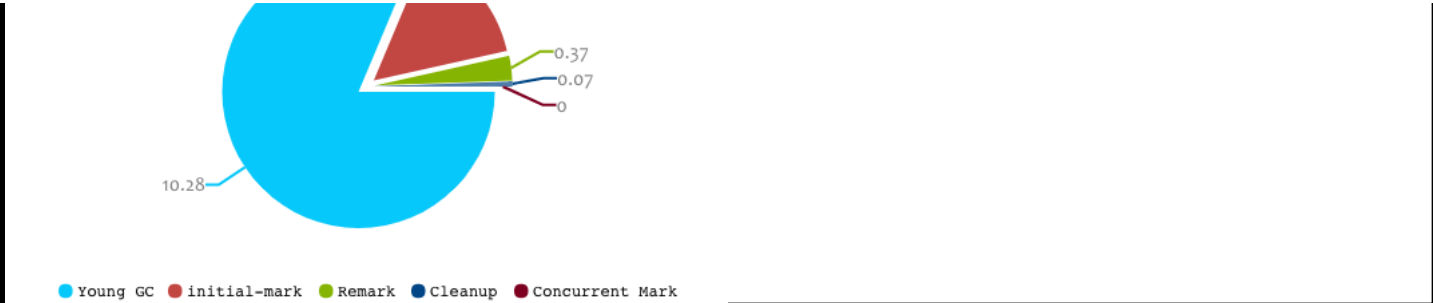
Interactive Graphs

(All graphs are zoomable)



G1 GC Statistics





	Concurrent Mark	Cleanup	Young GC	Remark	initial-mark	Total
Count ⓘ	10	10	59	10	11	100
Total GC Time ⓘ	0	66 ms	10 sec 280 ms	370 ms	1 sec 920 ms	12 sec 636 ms
Avg GC Time ⓘ	0	7 ms	174 ms	37 ms	175 ms	126 ms
Avg Time std dev	0	6 ms	65 ms	5 ms	36 ms	90 ms
Min/Max Time ⓘ	0 / 0	0 / 20 ms	0 / 470 ms	0 / 40 ms	0 / 260 ms	0 / 470 ms
Avg Interval Time ⓘ	32 sec 872 ms	32 sec 873 ms	49 sec 807 ms	32 sec 872 ms	35 sec 169 ms	43 sec 453 ms

⚙️ Object Stats

(These are good metrics to include in your performance reports)

Total created bytes ⓘ	59.42 gb
Total promoted bytes ⓘ	740 mb
Avg creation rate ⓘ	20.86 mb/sec
Avg promotion rate ⓘ	259 kb/sec

💧 Memory Leak ⓘ

No major memory leaks.

(**Note:** there are [8 flavours of OutOfMemoryErrors](https://tier1app.files.wordpress.com/2014/12/outofmemoryerror2.pdf) (<https://tier1app.files.wordpress.com/2014/12/outofmemoryerror2.pdf>). With GC Logs you can diagnose only 5

flavours of them (Java heap space, GC overhead limit exceeded, Requested array size exceeds VM limit, Permgen space, Metaspace). So in other words, your application could be still suffering from memory leaks, but need other tools to diagnose them, not just GC Logs.)

☰ Consecutive Full GC ⓘ

None.

■ Long Pause ⓘ

None.

🕒 Safe Point Duration ⓘ

(To learn more about SafePoint duration, [click here](https://blog.gceasy.io/2016/12/22/total-time-for-which-application-threads-were-stopped/) (https://blog.gceasy.io/2016/12/22/total-time-for-which-application-threads-were-stopped/))

Not Reported in the log.

❓ GC Causes ⓘ

(What events caused the GCs, how much time it consumed?)

Not reported in the log.

⌘ Tenuring Summary ⓘ

Not reported in the log.

📄 Command Line Flags ⓘ

Not reported in the log.

🏆 Become a DevOps champion in your organization

(Best practises/tools)

- ✓ Use **fastThread.io** (https://fastthread.io/) to analyze thread dumps, core dumps and hs_err_pid files
 - ✓ Do proactive Garbage Collection analysis on all your JVMs (not just one or two) [using the GC log analysis API](https://blog.gceasy.io/2016/06/18/garbage-collection-log-analysis-api/) 🚀 (https://blog.gceasy.io/2016/06/18/garbage-collection-log-analysis-api/)
 - ✓ Always [rotate the GC log files](https://blog.gceasy.io/2016/11/15/rotating-gc-log-files/) (https://blog.gceasy.io/2016/11/15/rotating-gc-log-files/) to prevent critical garbage collection data loss
 - ✓ Use '[Pro](http://gceasy.io/pricing.jsp)' [version](http://gceasy.io/pricing.jsp) (http://gceasy.io/pricing.jsp) for 10x fast, unlimited, secure usage
-

Do you like this report?